

UNIVERSIDADE DA CORUÑA

Programación de Sistemas

3.2 De monolito a CLEAN
Ejemplos

Carlos Vázquez Regueiro
carlos.vazquez.regueiro@udc.es

Repositorio

- Repositorio GIT con el código de ejemplo:
  - https://github.com/cvregueiro/PSICleanArch_v2
- Aplicación para listar grupos, bandas, artistas musicales en base a una búsqueda y ver sus conciertos
- Sigue el flujo de trabajo de GitFlow
  - Dos ramas:
    - **develop**: en la que se trabaja
    - **master**: contiene el código de las versiones que se publican
  - Nueva feature para cada nueva funcionalidad
  - Cuando se acabe, Pull Request (PR) a develop
  - Se aprueba y se junta con develop

Repositorio (y II)

- Las features están numeradas en el orden en que se aplican:
  - **01 - 09**: aplicación monolítica, todo está en las *Activities*
  - **10 - 16**: paso a una arquitectura *Clean* en capas
- Cada rama explica sus cambios en *docs/NN_nombre.md*
  - Las ramas 10 - 16 incluyen además su diagrama en cebolla
- El API de setlist.fm necesita una clave personal (API_KEY)
  - Se guarda en *local.properties*, que nunca se sube al repositorio
- Android Studio 2026.1, Gradle 9.8 (Kotlin DSL), Java 17
  - *minSdk* 24 y *targetSdk* 37

Feature **01_lista_de_artistas**

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/01_lista_de_artistas
- Código inicial creando una aplicación desde el wizard de Android Studio
- Se modificó el *.gitignore*
- Se añadió el *code_style_schema.xml* para el formateo automático de código
- En la *MainActivity* se creó una *RecyclerView* de *Strings* con su *Adapter* y *ViewHolder*
- Se activó **ViewBinding** en el *app/build.gradle.kts* (sustituye a ButterKnife)
  - Genera una clase por layout: *activity_main.xml* → *ActivityMainBinding*
- Se creó la clase *BaseActivity* que extiende de *AppCompatActivity*
  - Infla el *binding*, configura la *toolbar* y aplica el modo *edge-to-edge*

Feature **02_model_artist**

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/02_model_artist
- Se crea el modelo *Artist*
  - Con atributos *id* y *name*
  - Está en el paquete *domain*
- La *MainActivity*, en vez de tener un adapter de *Strings*, lo tendrá de *Artists*
- Los *Artists* aún se crean de forma manual (*hardcoded*) en la *MainActivity*
- Cada fila muestra el nombre del artista y, debajo, su *id*
  - El *ViewHolder* tiene un método *bind(Artist)* que encapsula cómo se pinta

Feature **03_retrofit_1** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/03_retrofit_1
- La idea es utilizar un API REST externa para conseguir la lista de artistas
- Vamos a utilizar la de MusicBrainz
  - Documentación: https://musicbrainz.org/doc/MusicBrainz_API
  - Ejemplo: https://musicbrainz.org/ws/2/artist/?query=artist:rancid&fmt=json
- Para acceder a esta API utilizaremos la librería Retrofit
  - Muy utilizada, altamente testada
  - Retrofit utiliza por debajo las librerías OkHttp y Gson
  - Las incluimos en el fichero *app/build.gradle.kts*
- Se añade el permiso *INTERNET* en el *AndroidManifest.xml*

Feature **03_retrofit_1** (II)

- Retrofit necesita de una interfaz que defina el API REST que vamos a utilizar
- La interfaz que hemos creado se llama *MusicBrainzAPI*
  - Contiene los *endpoints* del API que vamos a utilizar
  - Le indicamos el tipo de petición que es: GET, POST, PUT, PATCH o DELETE
  - Le indicamos los atributos y las cabeceras de la petición
- Estará en el paquete *data*
- Al ejecutar esta rama nos dará un error en tiempo de ejecución
  - Estamos llamando a un método síncrono de Retrofit para obtener datos de artistas del API REST
  - Al ser síncrono, Android produce una *android.os.NetworkOnMainThreadException*

Feature **03_retrofit_1** (y III)

- Que la llamada de Retrofit sea síncrona significa que la librería va a hacer la petición al API REST y se va a quedar esperando al resultado.
  - Esto bloqueará el hilo principal del teléfono haciendo que se quede “congelado”
  - Android no permite esto para evitar una mala experiencia de usuario
  - Cualquier acceso a red tiene que hacerse en otro hilo que no sea el principal
- Dos opciones:
  - Tener infraestructura para levantar nuevos *Threads* en estos casos
  - Utilizar las llamadas asíncronas que nos proporciona Retrofit
- MusicBrainz rechaza las peticiones sin un *User-Agent* que identifique a la app
  - Se añade con la anotación *@Headers* en la interfaz *MusicBrainzAPI*

Feature **04_retrofit_async** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/04_retrofit_async
- En esta feature vamos a utilizar los métodos asíncronos de Retrofit para evitar la *android.os.NetworkOnMainThreadException*
- Para ello se utilizará el método *enqueue*
- Necesitaremos un *Callback* que tendrá dos métodos que se ejecutarán cuando la petición haya acabado
  - *onResponse*: Para cuando haya ido bien la petición
  - *onFailure*: Para cuando falle la petición o no se pueda interpretar la respuesta
- Una vez llamado al método *enqueue*, al ser asíncrono, la ejecución del código sigue de manera secuencial, sólo se ejecutarán los métodos anteriores cuando el servidor responda

Feature **04_retrofit_async** (y II)

- Lo normal es tener un *ProgressBar* (o similar) para enseñar al usuario mientras se ejecuta la petición
  - Así él sabe que se está haciendo algo esperando
- Habrá veces en el que estas peticiones se harán de manera “invisible” al usuario
  - Con lo cual, no habrá que mostrarle nada
- Importante controlar bien el flujo de ejecución
  - Si lanzamos una petición asíncrona y cambiamos de *Activity* antes de que el servidor responda, el *Callback* actuaría sobre una *Activity* que ya no existe
- Esta rama mostrará un error (*Toast*) porque nuestro modelo (*Artist*) no coincide con el que devuelve el API

Feature **05_models** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/05_models
- La rama anterior daba error por lo siguiente
  - Nuestra interfaz *MusicBrainzAPI*, que define el API al que nos vamos a conectar, espera una lista de artistas (*List<Artist>*)
  - En cambio, la petición real devuelve un objeto, no una lista
  - Lo que declaramos en *MusicBrainzAPI* tiene que coincidir con lo que devuelve el API real
- Añadiremos al modelo la clase *SearchArtists*
  - Contendrá una lista de artistas: *List<Artist>*
  - Y un contador con el número de resultados: *int count*
  - Estos dos atributos (entre otros) los devuelve el API

Feature **05_models** (y II)

- El *id* de un artista es un MBID (un UUID), así que es un *String* en vez de un entero
- *@SerializedName* indica el nombre del atributo en el JSON; Gson ignora los que no se declaran
- La interfaz *MusicBrainzAPI* devuelve ahora un *SearchArtists*:

```
public interface MusicBrainzAPI {

    // https://musicbrainz.org/ws/2/artist/?query=artist:rancid&fmt=json
    @Headers("User-Agent: PSiCleanArch/" + BuildConfig.VERSION_NAME + " ( cvazquez@udc.es )")

    @GET("artist/")
    Call<SearchArtists> searchArtistByName(@Query("query") String query,
                                           @Query("fmt") String format);
}
```

Feature **06_retrofit_logging**

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/06_retrofit_logging
- Es importante que en el log veamos las peticiones que se hacen
- Hay que añadir un *Interceptor*
  - Los *Interceptors* "interceptan" la petición HTTP antes de ser emitida
  - Modifican la petición
  - Es típico que añadan parámetros y/o cabeceras de, por ejemplo, autenticación
- Para activar el logging de Retrofit se configura con la clase *HttpLoggingInterceptor*
- Se configuró para mostrar el log solo en entornos de desarrollo
  - Nivel *BODY* si *BuildConfig.DEBUG* y *NONE* en *release*

Feature **07_retrofit_headers** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/07_retrofit_headers
- Vamos a utilizar otro API para obtener los conciertos de los artistas
  - Será el API de setlist.fm
  - Documentación: https://api.setlist.fm/docs/1.0/index.html
- Hay que registrarse para obtener el API_KEY
  - El API_KEY se utiliza para la autenticación e identificación del usuario
  - Debe ir en todas las peticiones
  - Normalmente va en las cabeceras de la petición HTTP
- Para añadir estas cabeceras se utilizará un *Interceptor*
  - Porque tiene que ir en todas las peticiones HTTP

Feature **07_retrofit_headers** (y II)

- Se creó la clase *SetListFmApiInterceptor* (paquete *data.interceptor*)
  - Para añadir atributos comunes a todas las peticiones HTTP
    - El api_key para la autenticación: Header *x-api-key:API_KEY_DE_SETLIST.FM*
    - El formato de salida: Header *Accept: application/json*
- *MusicBrainzApiInterceptor* añade el *User-Agent* y sustituye a la anotación *@Headers*
- Como se hizo con el api de MusicBrainz: se creó la interfaz Retrofit *SetListFmAPI*
  - Y los objetos del modelo *SetList* (fecha del concierto) y *SetLists* (*List<SetList>*)
- El API_KEY se lee de *local.properties* como *BuildConfig.SETLISTFM_API_KEY*
  - Nunca se escribe en el código ni se sube al repositorio

Feature **08_search_query**

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/08_search_query
- Hasta ahora siempre se buscaba “rancid”: ahora el texto lo escribe el usuario
- En *activity_main.xml* se añadió un *TextInputEditText* (Material 3) y un botón **Buscar**
  - También una vista para los mensajes y un indicador de progreso
- La *MainActivity*:
  - Valida el texto: si está vacío, muestra un error en el campo sin llamar a la red
  - Oculta el teclado y lanza la búsqueda con el botón o con la tecla del teclado
  - Gestiona los estados de la UI: cargando, lista, sin resultados y error
- La consulta busca el texto como una frase: *artist:"texto"*

Feature **09_artist_gigs** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/09_artist_gigs
- Al pulsar un artista se abre una segunda *Activity*, *GigsActivity*, con sus conciertos (bolos)
  - Usa el API de setlist.fm y el *Interceptor* de la feature 07
- *GigsActivity* recibe el id y el nombre del artista en el *Intent*
  - Con un método de factoría, *newIntent(...)*, que documenta qué necesita
  - Muestra el nombre en la *toolbar*, con la flecha de volver
- *ArtistAdapter.OnArtistClickListener*: el adapter avisa a la *Activity* del artista pulsado
  - Sin conocer la *Activity*: se comunican con una interfaz (*listener*)

Feature **09_artist_gigs** (y II)

- Los objetos del modelo reproducen el JSON anidado de setlist.fm:
  - *SetList*, *Venue*, *City*, *Country* y *Tour*
- *GigAdapter* y *row_gig.xml* muestran fecha, sala, ciudad y país, y gira
- Casos especiales:
  - setlist.fm responde **404** cuando el artista no tiene conciertos: se trata como "sin resultados"
  - Si falta el API_KEY, se muestra un aviso en lugar de hacer la petición
  - El log oculta la cabecera *x-api-key* (*redactHeader*)
- Es el punto de partida de la fase *Clean*: la *Activity* hace de todo
  - Vista, acceso a red y transformación de datos

Feature **10_MVP**

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/10_MVP
- *MainActivity* pasa a ser *ArtistsActivity*, en el paquete *presentation.artist*
- En el paquete *presentation.artist.presenter* se añadieron:
  - *ArtistsPresenter*: Interfaz con las acciones que la *Activity* manda al *Presenter*
  - *ArtistsPresenterImp*: La implementación de la interfaz del *Presenter*
  - *ArtistsView*: Interfaz con las acciones que el *Presenter* le mandará a la *Activity*
    - La implementará la *Activity*
- Lo mismo para los conciertos: *GigsPresenter*, *GigsPresenterImp* y *GigsView*
- Así, la comunicación entre *Activity* y *Presenter* y viceversa queda completa

Feature **11_MVVP**

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/11_MVVP
- Para dotar de mayor independencia la capa de Vista y *Presenter* del resto de las capas inferiores, se utilizan modelos de vista (*UiModel*)
- Se creó la clase *ArtistUiModel* en *presentation.artist.model*
  - Contiene los atributos que va a necesitar la vista
  - Puede contener a varios objetos del modelo
- También necesitaremos un *mapper* para convertir objeto/s del modelo en *UiModel*
  - Lo hace la clase *ArtistUiModelMapper* del mismo paquete
- Lo mismo con *GigUiModel* y *GigUiModelMapper*: fecha formateada, “Ciudad, País”…
- Se llaman *UiModel* para no confundirlos con el *ViewModel* de Jetpack

Feature **12_services** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/12_services
- El *Presenter* sólo debe "presentar" lo que le devuelven los servicios
- Los servicios contendrán la lógica de negocio
  - Tratarán con los objetos del modelo
  - Estarán en el paquete *domain*
  - Esto hace que sólo podrán contener código Java, no Android
    - ¿Es posible? Lo veremos más adelante
- Las entidades *Artist* y *Gig* del dominio ya no tienen anotaciones de Gson
  - Los objetos del JSON pasan a *data* como DTOs, con sus *mappers* DTO → entidad

Feature **12_services** (II)

- Los servicios suelen ir asociados a un caso de uso
- Los servicios creados son:
  - *domain.artist.service.ArtistService*
    - Con un método de buscar artista pasándole un texto; devolverá una lista de artistas
  - *domain.gig.service.GigService*
    - Con un método de buscar los conciertos pasándole un identificador de artista; devolverá una lista de conciertos
- *TaskRunner* ejecuta el servicio en otro hilo y entrega el resultado en el principal
  - Sustituye a la *AsyncTask*, obsoleta desde Android 11

Feature **12_services** (y III)

- **Ejercicio: Crear las implementaciones de los servicios**
  - Irán en el mismo paquete que las interfaces
  - El nombre de las clases tendrán el sufijo *Imp*
    - Esto se hace para con solo ver el nombre saber qué hay dentro
  - Pasar la lógica de llamada a las APIs a los servicios
    - Ahora está en los *Presenters*
    - Moverla a los servicios
    - Los *Presenters* deberán llamar a los servicios a través del *TaskRunner*
  - Solución: *ArtistServiceImp* y *GigServiceImp* de esta rama

Feature **13_datasources** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/13_datasources
- La función de los servicios debe contener la lógica de negocio
- Esta lógica debe ser independiente de las fuentes de datos:
  - Base de datos
  - APIs externas
  - Memoria
  - …
- Crear otra capa, *datasources*, que hagan la función de “ir a buscar los datos”

Feature **13_datasources** (II)

- El acceso a las fuentes de información es, muchas veces, dependiente del sistema:
  - Hay que acceder a la memoria
  - Hay que acceder a red
  - …
- Por eso no puede ir en el paquete *domain*
  - Recordemos que era el paquete con código solo Java, no Android
- Irá en el paquete *data*
  - Ahí también estaban las interfaces de las APIs
  - Este paquete sí que es dependiente del sistema
    - Tiene que poder acceder a memoria, a red, etc...

Feature **13_datasources** (III)

- Pero las interfaces (recordemos que **siempre** utilizamos interfaces entre capas) podrán ir en el paquete *domain*
  - Las interfaces no tienen implementación
  - Por lo tanto, no tienen código, en este caso, Android
- En esta rama: *ArtistDatasource* (*domain.artist.datasource*) y *ArtistDatasourceImp* (*data.artist*)
- Como las llamadas a red tienen que ser en segundo plano, ¿cómo evitamos arrastrar el *Callback* por servicios y *datasources*?
  - El *datasource* usa Retrofit síncrono (*execute()*), como en 03_retrofit_1
  - Y el *Presenter* lanza la llamada en otro hilo con el *TaskRunner*

Feature **13_datasources** (y IV)

- **Ejercicio: Crear el datasource de los conciertos**
  - *GigServiceImp* aún usa Retrofit directamente: el dominio depende de la capa de datos
  - Crear la interfaz *GigDatasource* en *domain* y su implementación *GigDatasourceImp* en *data*
  - Trasladar el acceso al API de setlist.fm hasta el *datasource*
  - *GigServiceImp* recibirá el *datasource* por constructor
  - La *Activity* montará la cadena: servicio + *datasource*
  - Solución: *GigRepositoryImp* en la rama 14_clean_architecture

Feature **14_clean_architecture** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/14_clean_architecture
- Las ramas 12 y 13 crearon el dominio, pero con problemas:
  - *GigServiceImp*, dentro de *domain*, importaba Retrofit, OkHttp, los DTOs y *BuildConfig*
  - Un "servicio" con interfaz e *Imp* mezclaba varias acciones
  - Un *datasource* en el dominio le obliga a pensar de dónde salen los datos
- Se reorganiza la app en las tres capas de la *Clean Architecture*, en un único módulo:
  - *presentation*: *ui*, *presenter* y *uimodel*
  - *domain*: *model*, *repository*, *usecase* y *exception* (solo Java)
  - *data*: *repository* y *api*

Feature **14_clean_architecture** (II)

- Casos de uso: *SearchArtistsUseCase* y *GetArtistGigsUseCase*
  - Una clase por acción del usuario; sustituyen a los servicios
- Puertos: interfaces *ArtistRepository* y *GigRepository* en *domain.repository*
  - Las define el dominio y las implementa la capa de datos: *ArtistRepositoryImp* y *GigRepositoryImp*
  - Es la inversión de dependencias
- Excepciones del dominio: *DataUnavailableException* y *ConfigurationException*
  - Los repositorios traducen las *IOException* y los errores HTTP
- *ApiFactory* crea los clientes Retrofit y recibe la configuración como parámetros

Feature **14_clean_architecture** (y III)

- *GlobalApplication* + *AppContainer*: las dependencias se crean en un único sitio
  - Android crea la *GlobalApplication* una vez, antes que cualquier *Activity*
  - *AppContainer* es el único que conoce las implementaciones (*composition root*)
  - Las *Activities* piden ahí sus dependencias:
    - *((GlobalApplication) getApplication()).getContainer()*
- Tests unitarios en la JVM, sin emulador: casos de uso, *Presenters* y *mappers*
- La regla de dependencias se cumple… pero solo por disciplina
  - Nada impide escribir *import retrofit2.Call;* en el dominio

Feature **15_remote_local** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/15_remote_local
- Objetivo: que la aplicación funcione sin red
  - Con red: el repositorio pide datos al servidor y guarda una copia en fichero
  - Sin red: el repositorio responde con la copia guardada
- Hay dos fuentes de datos y el repositorio decide cuál usar
  - Ni el dominio ni la presentación cambian
- Nuevos paquetes en *data*:
  - *remote*: *ArtistRemoteDataSource*, *GigRemoteDataSource* (Retrofit)
  - *local*: *FileArtistDataSource*, *FileGigDataSource* y *JsonFileStore*
  - *network*: *NetworkChecker* (¿hay red?) y *AndroidNetworkChecker*

Feature **15_remote_local** (II)

- El fichero guarda los mismos DTOs que devolvió el servidor
  - Se reutiliza el mismo *mapper* DTO → entidad para las dos fuentes
- Los ficheros van en *getFilesDir()/offline*, no en la caché: Android puede vaciarla
- *NetworkChecker* es una interfaz: el repositorio no depende de Android
  - *AndroidNetworkChecker* la implementa con el *ConnectivityManager*
  - En los tests se simula con una lambda: *() -> true*
- Sin red ni copia guardada: *DataUnavailableException* y la vista muestra el error
- Para probarlo, desactivar Wi-Fi y datos móviles: *adb shell svc wifi disable*

Feature **15_remote_local** (y III)

- **Ejercicio: Usar la copia local también cuando el servidor falla**
  - MusicBrainz responde a menudo con un error 503 (servidor ocupado)
  - Ahora, con red, si el servidor falla se muestra un error aunque haya copia guardada
  - Modificar *ArtistRepositoryImp* y *GigRepositoryImp* para leer la copia en caso de que falle la red
    - La política de qué fuente usar está solo en el repositorio
  - Añadir un test que lo compruebe en *ArtistRepositoryImpTest*

Feature **16_modules** (I)

- https://github.com/cvregueiro/PSICleanArch_v2/tree/feature/16_modules
- En las ramas 14 y 15 la arquitectura es correcta, pero solo por disciplina
  - Todo está en el módulo *:app*: nada impide importar Retrofit o Android en el dominio
- Se separan las capas en tres módulos Gradle:
  - *:domain*: librería Java pura, sin dependencias
  - *:data*: librería Java con Retrofit, OkHttp y Gson; depende de *:domain*
  - *:app*: Android; presentación, *GlobalApplication* y *AppContainer*
- Romper la regla de dependencias pasa a ser un error de compilación

Feature **16_modules** (II)

- Los paquetes son los mismos que en la rama 15
  - Los módulos no crean la arquitectura: la protegen
- *AndroidNetworkChecker* usa Android: no puede estar en *:data* y pasa a *:app*
  - Implementa la interfaz *NetworkChecker* de *:data*
- *RepositoryFactory*: única entrada pública a *:data*
  - Retrofit es *implementation* en *:data*, así que *:app* no lo ve
  - Devuelve solo los puertos del dominio
- Compilación incremental y tests de *:domain* y *:data* sin Android

Feature **16_modules** (y III)

- **Ejercicio: Comprobar que el compilador hace cumplir la regla de dependencias**
  - Añadir *import retrofit2.Call;* en *SearchArtistsUseCase*: ¿compila?
  - Añadir *import android.util.Log;* en *ArtistRepositoryImp*: ¿compila?
  - Hacer lo mismo en la rama 15_remote_local y comparar
  - Explicar la diferencia entre *api* e *implementation* en *data/build.gradle.kts*